

Practical: 2

Implementation and Time analysis of linear and binary search algorithm

Aim: To Implement Linear search and binary search and Time analysis their algorithms.

Algorithm:

Linear Search (Algorithm)

- * Start from the first element
- * Compare each element with the key
- * If found $\rightarrow$ return position
- * If not $\rightarrow$ move to next
- * End after last element (not found)

Binary Search

- * work on sorted array
- * set low and high pointers
- * Find $\text{mid} = (\text{low} + \text{high}) / 2$
- * If $\text{key} = \text{mid} \rightarrow$ return position
- * If $\text{key} < \text{mid} \rightarrow$ search left half
- * If $\text{key} > \text{mid} \rightarrow$ search right half
- * Repeat until $\text{low} > \text{high}$ (not found)

Program:

```
#include <iostream>
using namespace std;

// Linear Search
int linear(int array[], int s, int t) {
    for (int i = 0; i < s; i++) {
        if (array[i] == t) {
            return i;
        }
    }
    return -1;
}

// Binary Search
int binary(int array[], int s, int t) {
    int l = 0;
    int h = s - 1;

    while (l <= h) {
        int m = (l + h) / 2;

        if (array[m] == t) {
            return m;
        }
        else if (array[m] < t) {
            l = m + 1;
        }
        else {
            h = m - 1;
        }
    }

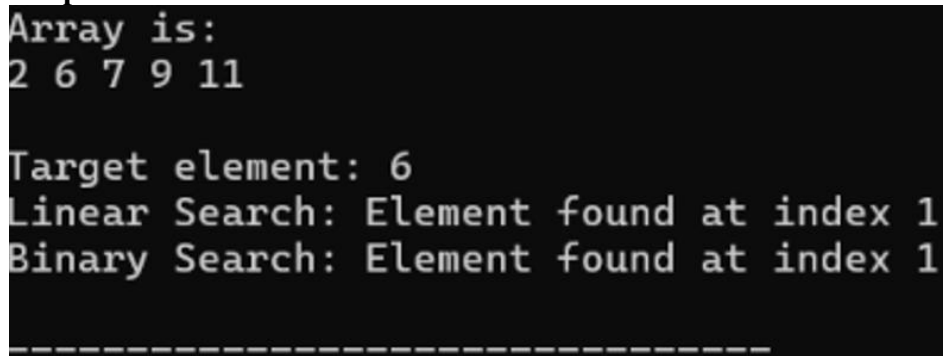
    return -1;
}

// Display Array
void display(int array[], int n) {
    cout << "Array is:\n";

    for (int i = 0; i < n; i++) {
        cout << array[i] << " ";
    }

    cout << endl;
}
```

```
int main() {  
  
    int array[] = {2, 6, 7, 9, 11};  
    int n = sizeof(array) / sizeof(array[0]);  
  
    int t = 6;  
  
    display(array, n);  
  
    int f = linear(array, n, t);  
    int g = binary(array, n, t);  
  
    cout << "\nTarget element: " << t << endl;  
  
    if (f != -1) {  
        cout << "Linear Search: Element found at index " << f << endl;  
    }  
    else {  
        cout << "Linear Search: Element not found" << endl;  
    }  
  
    if (g != -1) {  
        cout << "Binary Search: Element found at index " << g << endl;  
    }  
    else {  
        cout << "Binary Search: Element not found" << endl;  
    }  
  
    return 0;  
}
```

Output:

```
Array is:  
2 6 7 9 11  
  
Target element: 6  
Linear Search: Element found at index 1  
Binary Search: Element found at index 1  
-----
```



Time Complexity:

Time complexity: Time to search in array size n
 $T(n) = T(n-1) + O(1)$

$$T(n) = T(n-1) + 1 = T(n-2) + 1 + 1 + \dots + T(1) + (n-1)$$

Best case: $T(1) = O(1)$

Final: $T(n) = O(n)$

* Best case: $O(1)$

* Worst case: $O(n)$

* Average case: $O(n)$

Time complexity:

let $T(n)$ = time to search in array of size n .
 for checking middle element constant $O(1)$
 search half the array $\rightarrow n/2$

$$T(n) = T\left(\frac{n}{2}\right) + 1 = T\left(\frac{n}{4}\right) + 1 + 1 = \dots + T(1) + \log_2 n$$

Best case: $T(1) = O(1)$

$$T(n) = O(\log n)$$

Best case = $O(1)$

Average case = $O(\log n)$

Worst case = $O(\log n)$

Conclusion: Hence From this practical of linear search, Binary search the algorithm and programs have executed successfully. And Time complexity has categorised into three ways as Best, average and worst.